消息队列 / Kafka / Redis

原理详解与常见八股文

涵盖消息队列核心原理、Kafka 架构与高性能设计、Redis 数据结构与集群方案

以及高频面试题解析

2026 年 3 月 9 日

目 录

第一章 消息队列原理

1.1 什么是消息队列

1.2 消息队列的核心作用

1.3 消息队列的通信模型

1.4 消息队列的关键概念

1.5 消息队列的可靠性保障

1.6 主流消息队列对比

第二章 Kafka 原理详解

2.1 Kafka 概述

2.2 Kafka 架构

2.3 Kafka 核心机制

2.4 Kafka 常见八股文（7 题）

第三章 Redis 原理详解

3.1 Redis 概述

3.2 Redis 数据结构与底层编码

3.3 Redis 线程模型

3.4 Redis 持久化机制

3.5 Redis 内存管理与淘汰策略

3.6 Redis 集群方案

3.7 Redis 常见八股文（8 题）

第一章 消息队列原理

1.1 什么是消息队列

消息队列（Message Queue，简称 MQ）是一种应用程序间的异步通信机制。它通过在发送方和接收方之间引入一个中间代理（Broker），将消息的生产和消费解耦。发送方将消息投递到队列中，接收方从队列中拉取或被推送消息，双方无需同时在线，也无需知道彼此的存在。

消息队列的核心思想源自生产者-消费者模型：生产者负责产生数据并放入缓冲区，消费者从缓冲区取出数据进行处理。队列充当了缓冲区的角色，使得生产和消费的速率可以不一致，从而实现系统间的松耦合。

1.2 消息队列的核心作用

1.2.1 异步处理

在同步调用模式下，调用方需要等待被调用方处理完成后才能继续执行，这会导致响应时间变长。消息队列允许将耗时操作（如发送邮件、生成报表、写入日志等）异步化。主流程只需将消息投递到队列即可返回，后续由消费者异步完成处理，从而大幅降低接口响应时间。

1.2.2 应用解耦

在微服务架构中，服务之间的直接调用会形成强依赖关系。当下游服务不可用时，上游服务也会受到影响。通过消息队列，上游服务只需将事件发布到队列，下游服务订阅并消费，两者之间不再有直接的调用关系。即使下游服务暂时不可用，消息也会在队列中暂存，待服务恢复后继续消费。

1.2.3 流量削峰

在高并发场景下（如秒杀、抢购），瞬时请求量可能远超系统处理能力。消息队列可以作为缓冲层，将突发流量暂存在队列中，消费者按照自身处理能力匀速消费，避免系统被瞬时流量压垮。这种模式也被称为“削峰填谷”。

1.2.4 数据分发

一条消息可以被多个消费者订阅和消费，实现一对多的数据分发。例如，一个订单创建事件可以同时触发库存扣减、积分增加、短信通知等多个下游操作，而订单服务无需逐一调用这些下游服务。

1.3 消息队列的通信模型

1.3.1 点对点模型（P2P）

在点对点模型中，消息被发送到一个特定的队列，每条消息只能被一个消费者消费。消费者从队列中主动拉取消息，消费完成后消息从队列中移除。这种模型适用于任务分发场景，确保每个任务只被处理一次。

1.3.2 发布/订阅模型（Pub/Sub）

在发布/订阅模型中，消息被发布到一个主题（Topic），所有订阅了该主题的消费者都能收到消息的副本。这种模型适用于事件广播场景，一条消息可以被多个消费者独立消费。Kafka 和 RocketMQ 主要采用这种模型。

1.4 消息队列的关键概念

概念	说明
Producer（生产者）	消息的发送方，负责创建消息并投递到 Broker
Consumer（消费者）	消息的接收方，负责从 Broker 拉取或接收消息并处理
Broker（代理）	消息队列的服务端，负责存储和转发消息
Topic（主题）	消息的逻辑分类，生产者向 Topic 发送消息，消费者订阅 Topic
Queue（队列）	消息的物理存储单元，保证消息的先进先出（FIFO）顺序
Consumer Group	消费者组，组内消费者共同消费一个 Topic，实现负载均衡
Offset（偏移量）	消费者在队列中的消费位置标记，用于记录消费进度
ACK（确认机制）	消费者处理完消息后向 Broker 发送确认，防止消息丢失

1.5 消息队列的可靠性保障

1.5.1 消息持久化

为防止 Broker 宕机导致消息丢失，消息队列通常支持将消息持久化到磁盘。生产者发送消息时可以选择同步刷盘（消息写入磁盘后才返回确认）或异步刷盘（消息写入内存后即返回确认，后台异步刷盘）。同步刷盘可靠性更高但性能较低，异步刷盘性能更好但存在少量消息丢失的风险。

1.5.2 消息确认机制

消费者从队列中获取消息后，需要向 Broker 发送 ACK 确认。如果 Broker 在超时时间内未收到 ACK，会认为消费失败并重新投递消息。这种机制确保消息至少被消费一次（At Least Once），但也可能导致重复消费，因此消费者端需要做好幂等处理。

1.5.3 消息重试与死信队列

当消费者处理消息失败时，消息队列通常支持自动重试机制。消息会被重新投递给消费者，重试次数可配置。如果经过多次重试仍然失败，消息会被转移到死信队列（Dead Letter Queue，DLQ），由人工介入处理，避免问题消息阻塞正常消费。

1.6 主流消息队列对比

特性	Kafka	RocketMQ	RabbitMQ	ActiveMQ
开发语言	Scala/Java	Java	Erlang	Java
单机吞吐量	百万级/秒	十万级/秒	万级/秒	万级/秒
消息延迟	毫秒级	毫秒级	微秒级	毫秒级
可用性	非常高（分布式）	非常高（分布式）	高（镜像集群）	高（主从）
消息可靠性	可配置（0 丢失）	可配置（0 丢失）	高	较低
功能特性	简单，主要用于大数据	功能丰富，事务消息	功能完善，路由灵活	功能完善，协议丰富

适用场景	日志收集、大数据流处理	电商、金融等业务场景	中小规模、复杂路由	传统企业应用

第二章 Kafka 原理详解

2.1 Kafka 概述

Apache Kafka 是一个分布式流处理平台，最初由 LinkedIn 开发，后捐赠给 Apache 基金会。Kafka 以高吞吐量、低延迟、高可用和可扩展性著称，广泛应用于日志收集、消息系统、流处理、事件溯源等场景。Kafka 的设计哲学是将消息持久化到磁盘，并通过顺序读写和零拷贝等技术实现极高的 I/O 性能。

2.2 Kafka 架构

2.2.1 核心组件

- **Broker**：Kafka 集群中的每个服务器节点称为一个 Broker。每个 Broker 负责存储部分分区的数据，多个 Broker 组成一个 Kafka 集群。Broker 之间通过 ZooKeeper（或 KRaft）协调元数据。
- **Topic**：消息的逻辑分类。生产者向 Topic 发送消息，消费者订阅 Topic 消费消息。一个 Topic 可以有多个分区。
- **Partition（分区）**：Topic 的物理分片。每个 Partition 是一个有序的、不可变的消息序列，消息在 Partition 内按照 Offset 严格有序。分区是 Kafka 实现并行处理和水平扩展的基础。
- **Replica（副本）**：每个 Partition 可以配置多个副本，分布在不同的 Broker 上。副本分为 Leader 和 Follower，Leader 负责读写请求，Follower 从 Leader 同步数据，提供容错能力。
- **Producer（生产者）**：消息的发送方。Producer 将消息发送到指定 Topic 的某个 Partition，可以通过 Key 的哈希值或自定义分区策略决定消息发往哪个 Partition。

- **Consumer（消费者）**：消息的接收方。Consumer 从 Broker 拉取消息进行处理。多个 Consumer 可以组成 Consumer Group，组内的 Consumer 共同消费一个 Topic 的所有 Partition。
- **ZooKeeper / KRaft**：负责集群元数据管理、Broker 注册、Leader 选举等。Kafka 2.8+ 引入 KRaft 模式，逐步去除对 ZooKeeper 的依赖。

2.2.2 数据存储结构

Kafka 的数据存储以 Partition 为单位。每个 Partition 对应磁盘上的一个目录，目录下包含多个 Segment 文件。每个 Segment 由一个 .log 文件（存储消息数据）和一个 .index 文件（存储偏移量索引）组成。当 .log 文件达到配置的大小阈值时，会创建新的 Segment。

这种分段存储的设计有两个好处：一是便于过期数据的清理（直接删除整个 Segment 文件），二是通过稀疏索引加速消息查找（先通过二分查找定位 Segment，再在 Segment 内通过索引定位消息）。

2.3 Kafka 核心机制

2.3.1 生产者发送机制

Kafka 生产者采用批量发送和异步发送的方式提高吞吐量。生产者内部维护一个消息缓冲区（RecordAccumulator），消息先写入缓冲区，由 Sender 线程批量发送到 Broker。发送时可以配置三种确认模式：

- **acks=0**：生产者不等待 Broker 确认，发送即忘。吞吐量最高，但可能丢失消息。
- **acks=1**：生产者等待 Leader 写入成功后返回确认。Leader 宕机时可能丢失未同步到 Follower 的消息。
- **acks=all（-1）**：生产者等待所有 ISR（In-Sync Replicas）副本都写入成功后返回确认。可靠性最高，但延迟较大。

2.3.2 消费者消费机制

Kafka 消费者采用拉取（Pull）模式从 Broker 获取消息。消费者通过维护 Offset 来记录消费进度。Offset 可以自动提交（定时提交）或手动提交（消费完成后显式提交）。

Consumer Group 是 Kafka 实现负载均衡的核心机制。一个 Consumer Group 内的多个 Consumer 共同消费一个 Topic 的所有 Partition，每个 Partition 只能被组内的一个 Consumer 消费。当 Consumer 数量发生变化时，会触发 Rebalance（重平衡），重新分配 Partition 与 Consumer 的对应关系。

2.3.3 副本机制与 ISR

Kafka 通过副本机制保证数据的高可用。每个 Partition 有一个 Leader 副本和多个 Follower 副本。所有的读写请求都由 Leader 处理，Follower 从 Leader 拉取数据进行同步。

ISR（In-Sync Replicas）是与 Leader 保持同步的副本集合。当 Follower 的同步延迟超过阈值时，会被移出 ISR。当 Leader 宕机时，Kafka 会从 ISR 中选举新的 Leader，确保新 Leader 拥有最新的数据。如果 ISR 为空，可以配置是否允许从非 ISR 副本中选举 Leader（unclean.leader.election.enable），这是可用性和一致性之间的权衡。

2.3.4 高性能设计

Kafka 之所以能实现极高的吞吐量，主要依赖以下几个关键技术：

- **顺序写磁盘：**Kafka 将消息追加写入到日志文件末尾，避免了随机写磁盘的性能开销。顺序写磁盘的速度可以达到 600MB/s，远超随机写的 100KB/s。
- **Page Cache：**Kafka 大量利用操作系统的页缓存（Page Cache），将磁盘 I/O 转化为内存操作。写入时消息先写入 Page Cache，由操作系统异步刷盘；读取时如果数据在 Page Cache 中则直接返回，避免磁盘 I/O。
- **零拷贝（Zero Copy）：**Kafka 使用 sendfile 系统调用，将数据直接从 Page Cache 传输到网络 Socket，跳过了用户态的数据拷贝，减少了两次数据拷贝和两次上下文切换。
- **批量处理：**生产者端批量发送消息，Broker 端批量写入磁盘，消费者端批量拉取消息。批量操作摊薄了网络和磁盘 I/O 的开销。

- **分区并行：**Topic 被分为多个 Partition，不同 Partition 可以分布在不同 Broker 上，实现读写的并行处理和水平扩展。

2.4 Kafka 常见八股文

Q1：Kafka 如何保证消息不丢失？

生产者端：设置 acks=all，确保消息被所有 ISR 副本确认；设置 retries > 0，开启重试机制；设置 max.in.flight.requests.per.connection=1（或使用幂等生产者），防止消息乱序。

Broker 端：设置 replication.factor >= 3，保证足够的副本数；设置 min.insync.replicas >= 2，确保至少有 2 个副本同步成功；关闭 unclean.leader.election.enable，防止数据不完整的副本成为 Leader。

消费者端：关闭自动提交 Offset（enable.auto.commit=false），改为手动提交，确保消息处理完成后再提交 Offset。

Q2：Kafka 如何保证消息的顺序性？

Kafka 只保证单个 Partition 内的消息有序。要实现全局有序，可以将 Topic 设置为只有一个 Partition，但这会牺牲并行性能。在实际场景中，通常通过设置消息的 Key（如订单 ID），让同一 Key 的消息发送到同一 Partition，从而保证同一业务实体的消息有序。同时需要设置 max.in.flight.requests.per.connection=1 或开启幂等生产者，防止重试导致的消息乱序。

Q3：Kafka 如何处理消息重复消费？

Kafka 默认提供 At Least Once 语义，即消息至少被消费一次，可能存在重复消费。处理重复消费的核心思路是消费者端做幂等处理。常见方案包括：利用数据库唯一键约束去重；使用 Redis 的 SETNX 命令记录已消费的消息 ID；在业务逻辑中通过状态机判断是否已处理。Kafka 0.11+

支持幂等生产者和事务消息，可以实现 Exactly Once 语义。

Q4：Kafka 的 Rebalance 机制是什么？什么时候触发？

Rebalance 是 Consumer Group 内重新分配 Partition 与 Consumer 对应关系的过程。触发条件包括：Consumer Group 中有新的 Consumer 加入或已有 Consumer 离开（主动退出或心跳超时）；订阅的 Topic 的 Partition 数量发生变化；Consumer 订阅的 Topic 列表发生变化。Rebalance 期间，所有 Consumer 会暂停消费，可能导致短暂的消费延迟。可以通过合理设置 session.timeout.ms 和 heartbeat.interval.ms 来减少不必要的 Rebalance。

Q5：Kafka 的 ISR 机制是什么？

ISR（In-Sync Replicas）是与 Leader 保持同步的副本集合。Follower 副本通过不断从 Leader 拉取数据来保持同步。如果 Follower 的同步延迟超过 replica.lag.time.max.ms（默认 10 秒），则会被移出 ISR。当 Leader 宕机时，新的 Leader 从 ISR 中选举产生。ISR 机制是 Kafka 在可用性和一致性之间的平衡：它不要求所有副本都同步（避免最慢副本拖慢整体），但保证选举出的新 Leader 数据是最新的。

Q6：Kafka 为什么性能这么高？

Kafka 的高性能主要来自五个方面：顺序写磁盘避免随机 I/O；利用操作系统 Page Cache 将磁盘操作转化为内存操作；使用 sendfile 零拷贝技术减少数据拷贝次数；生产者、Broker、消费者三端都采用批量处理；Topic 分区机制实现并行读写和水平扩展。

Q7：Kafka 中 ZooKeeper 的作用是什么？KRaft 模式有什么优势？

在传统架构中，ZooKeeper 负责 Broker 注册与发现、Controller 选举、Topic 和 Partition 元数据管理、Consumer Group 的 Offset 存储（旧版本）等。KRaft 模式（Kafka 2.8+）使用

Kafka 自身的 Raft 协议替代 ZooKeeper，优势包括：简化部署和运维（不再需要独立的 ZooKeeper 集群）；减少元数据同步的延迟；提升集群的可扩展性（ZooKeeper 在大规模集群下是瓶颈）。

第三章 Redis 原理详解

3.1 Redis 概述

Redis（Remote Dictionary Server）是一个开源的、基于内存的高性能键值存储系统。它支持丰富的数据结构，包括 String、Hash、List、Set、Sorted Set、Bitmap、HyperLogLog、Stream 等。Redis 以单线程模型和基于内存的操作实现了极高的读写性能（单机可达 10 万+ QPS），广泛应用于缓存、分布式锁、计数器、排行榜、消息队列等场景。

3.2 Redis 数据结构与底层编码

Redis 的每种数据类型在底层都有多种编码实现，Redis 会根据数据的大小和数量自动选择最优的编码方式，以在内存占用和访问性能之间取得平衡。

数据类型	底层编码	适用场景
String	int / embstr / raw	缓存、计数器、分布式锁
Hash	ziplist / hashtable（7.0+ 用 listpack）	对象属性存储、用户信息
List	ziplist / linkedlist（3.2+ 用 quicklist）	消息队列、最新列表
Set	intset / hashtable	标签、共同好友、去重
Sorted Set	ziplist / skiplist + hashtable	排行榜、延迟队列
Bitmap	String（SDS）	签到、在线状态、布隆过滤器
HyperLogLog	稀疏/稠密编码	UV 统计、基数估算
Stream	Rax + listpack	消息队列（支持消费者组）

3.3 Redis 线程模型

3.3.1 单线程模型

Redis 的核心命令处理采用单线程模型，即所有客户端的请求都由一个主线程串行处理。这种设计避免了多线程的上下文切换和锁竞争开销，同时保证了命令执行的原子性。Redis 之所以单线程还能如此高效，主要原因包括：所有数据都在内存中操作，内存访问速度极快；使用 I/O 多路

复用（epoll/kqueue）同时监听多个客户端连接；单线程避免了锁竞争和上下文切换的开销；数据结构经过精心优化，操作复杂度低。

3.3.2 多线程演进（Redis 6.0+）

Redis 6.0 引入了多线程 I/O，但命令执行仍然是单线程的。多线程主要用于网络数据的读取和写入（即 Socket 读写），因为在高并发场景下，网络 I/O 成为了性能瓶颈。主线程负责命令的解析和执行，I/O 线程负责读取请求数据和写回响应数据，两者通过无锁队列协作。

3.4 Redis 持久化机制

3.4.1 RDB（Redis Database）

RDB 是 Redis 的快照持久化方式。它在指定的时间间隔内，将内存中的数据集快照写入磁盘，生成一个紧凑的二进制文件（dump.rdb）。RDB 的优点是文件紧凑、恢复速度快，适合做数据备份和灾难恢复。缺点是无法做到实时持久化，在两次快照之间如果 Redis 宕机，会丢失这段时间内的数据。

RDB 的触发方式包括：手动执行 SAVE（阻塞主线程）或 BGSAVE（fork 子进程异步执行）命令；配置自动触发条件（如 save 900 1 表示 900 秒内至少有 1 次修改则触发）；主从复制时主节点自动触发。BGSAVE 利用操作系统的 Copy-On-Write（COW）机制，fork 出的子进程与主进程共享内存页，只有在主进程修改数据时才会复制对应的内存页，因此 fork 操作本身非常快。

3.4.2 AOF（Append Only File）

AOF 以日志追加的方式记录每一条写命令。Redis 重启时通过重放 AOF 文件中的命令来恢复数据。AOF 的优点是数据安全性更高，可以配置不同的同步策略：always（每条命令都同步到磁盘，最安全但最慢）、everysec（每秒同步一次，推荐配置，最多丢失 1 秒数据）、no（由操作系统决定何时同步，性能最好但最不安全）。

AOF 文件会随着时间增长变得很大，Redis 提供了 AOF 重写（Rewrite）机制来压缩文件。重写时，Redis fork 一个子进程，根据当前内存中的数据状态生成一个新的 AOF 文件，用最少的命

令表达当前数据集。重写期间主进程的新写命令会同时追加到旧 AOF 文件和重写缓冲区，重写完成后将缓冲区的命令追加到新文件并替换旧文件。

3.4.3 混合持久化（Redis 4.0+）

混合持久化结合了 RDB 和 AOF 的优点。在 AOF 重写时，先将当前数据以 RDB 格式写入新的 AOF 文件头部，然后将重写期间的增量命令以 AOF 格式追加到文件尾部。这样既保证了恢复速度（RDB 部分快速加载），又保证了数据完整性（AOF 部分记录增量）。

3.5 Redis 内存管理与淘汰策略

当 Redis 使用的内存达到 maxmemory 配置的上限时，会触发内存淘汰策略。Redis 提供了 8 种淘汰策略：

策略	说明
noeviction	不淘汰，新写入操作直接报错（默认策略）
allkeys-lru	从所有 Key 中淘汰最近最少使用的 Key
allkeys-lfu	从所有 Key 中淘汰最不经常使用的 Key（Redis 4.0+）
allkeys-random	从所有 Key 中随机淘汰
volatile-lru	从设置了过期时间的 Key 中淘汰最近最少使用的
volatile-lfu	从设置了过期时间的 Key 中淘汰最不经常使用的
volatile-random	从设置了过期时间的 Key 中随机淘汰
volatile-ttl	从设置了过期时间的 Key 中淘汰 TTL 最小的

Redis 的 LRU 算法并非严格的 LRU，而是近似 LRU。它通过随机采样（默认采样 5 个 Key）并淘汰其中最久未使用的 Key 来近似实现 LRU 效果，这种方式在内存开销和淘汰效果之间取得了良好的平衡。

3.6 Redis 集群方案

3.6.1 主从复制

主从复制是 Redis 高可用的基础。主节点（Master）负责读写，从节点（Slave）从主节点同步数据，提供读服务。复制过程分为全量复制（首次连接或断线时间过长时，主节点生成 RDB 快照发送给从节点）和增量复制（正常运行时，主节点将写命令通过复制缓冲区同步给从节点）。

3.6.2 哨兵模式（Sentinel）

哨兵模式在主从复制的基础上增加了自动故障转移能力。Sentinel 节点负责监控主从节点的健康状态，当主节点不可用时，自动将一个从节点提升为新的主节点，并通知其他从节点和客户端。Sentinel 通过多数投票机制判断主节点是否真正下线，避免误判。

3.6.3 Cluster 模式

Redis Cluster 是 Redis 官方的分布式方案，通过数据分片实现水平扩展。Cluster 将整个数据空间划分为 16384 个哈希槽（Hash Slot），每个节点负责一部分槽。客户端通过 CRC16(key) % 16384 计算 Key 所属的槽，然后路由到对应的节点。Cluster 支持在线扩缩容（迁移槽和数据）和自动故障转移（每个主节点有从节点作为备份）。

3.7 Redis 常见八股文

Q1：Redis 为什么这么快?

Redis 的高性能主要来自以下几个方面：基于内存操作，内存的读写速度远快于磁盘；单线程模型避免了多线程的上下文切换和锁竞争；使用 I/O 多路复用技术（epoll/kqueue）高效处理大量并发连接；数据结构经过精心设计和优化，如 SDS、ziplist、skiplist 等都针对特定场景做了性能优化。

Q2：缓存穿透、缓存击穿、缓存雪崩的区别和解决方案?

缓存穿透：查询一个不存在的数据，缓存和数据库都没有，每次请求都会打到数据库。解决方案

包括：缓存空值（设置较短的过期时间）；使用布隆过滤器在缓存层前置过滤；对请求参数做合法性校验。

缓存击穿：某个热点 Key 在缓存过期的瞬间，大量并发请求同时打到数据库。解决方案包括：设置热点 Key 永不过期；使用互斥锁（如 Redis 的 SETNX），只允许一个线程查询数据库并回填缓存；使用逻辑过期（缓存不设置 TTL，在值中记录逻辑过期时间，过期后异步更新）。

缓存雪崩：大量 Key 在同一时间过期，或者 Redis 服务宕机，导致大量请求直接打到数据库。解决方案包括：给 Key 的过期时间加上随机值，避免同时过期；使用 Redis 集群保证高可用；设置多级缓存（本地缓存 + Redis）；对数据库访问加限流和降级。

Q3：Redis 如何实现分布式锁？

最基本的方式是使用 SET key value NX EX 命令，NX 保证只有 Key 不存在时才能设置成功（互斥性），EX 设置过期时间防止死锁。释放锁时需要先判断锁的持有者（通过 value 中存储的唯一标识），再删除 Key，这两步需要通过 Lua 脚本保证原子性。在集群环境下，单节点的分布式锁可能因为主从切换导致锁丢失，可以使用 RedLock 算法：向 N 个独立的 Redis 节点同时加锁，当超过半数节点加锁成功且总耗时小于锁的过期时间时，才认为加锁成功。

Q4：Redis 的过期删除策略是什么？

Redis 采用惰性删除和定期删除相结合的策略。惰性删除是指在访问 Key 时才检查是否过期，如果过期则删除。这种方式对 CPU 友好，但可能导致大量过期 Key 长时间占用内存。定期删除是指 Redis 每隔一段时间（默认每秒 10 次）随机抽取一批设置了过期时间的 Key 进行检查，删除其中已过期的。如果过期 Key 的比例超过 25%，则继续抽取检查，直到比例降到 25% 以下或超时。两种策略结合使用，在 CPU 和内存之间取得平衡。

Q5：Redis 事务和 Lua 脚本的区别？

Redis 事务通过 MULTI/EXEC 命令实现，将多个命令打包后一次性执行，保证原子性（要么全部执行，要么全部不执行）。但 Redis 事务不支持回滚，如果某条命令执行失败，后续命令仍会继续执行。Lua 脚本在 Redis 中以原子方式执行，脚本执行期间不会被其他命令打断。相比事务，Lua 脚本更灵活，支持条件判断和循环，可以实现更复杂的原子操作（如 CAS 操作、分布式锁的释放等）。在实际开发中，推荐使用 Lua 脚本替代事务。

Q6：Redis 主从复制的原理?

Redis 主从复制分为三个阶段。第一阶段是建立连接：从节点向主节点发送 PSYNC 命令请求同步。第二阶段是全量复制：主节点执行 BGSAVE 生成 RDB 快照，发送给从节点；从节点清空旧数据，加载 RDB 文件；主节点将 RDB 生成期间的写命令缓存在复制缓冲区，RDB 发送完成后再将缓冲区的命令发送给从节点。第三阶段是增量复制：全量复制完成后，主节点将后续的写命令实时同步给从节点。如果从节点断线重连，会通过复制偏移量（Offset）和复制积压缓冲区（Replication Backlog）判断是否可以增量复制，如果缺失的数据在积压缓冲区中则增量复制，否则触发全量复制。

Q7：Redis 的 BigKey 问题如何处理?

BigKey 是指 Key 对应的 Value 占用内存过大（如 String 类型超过 10KB，集合类型元素超过 5000 个）。BigKey 的危害包括：内存分布不均导致数据倾斜；操作 BigKey 耗时长，阻塞主线程；删除 BigKey 可能导致长时间阻塞。处理方案包括：拆分 BigKey（如将大 Hash 拆分为多个小 Hash）；使用 UNLINK 命令异步删除（Redis 4.0+）；使用 SCAN 命令分批遍历和处理；在设计阶段就避免产生 BigKey。可以通过 redis-cli --bigkeys 命令或 MEMORY USAGE 命令发现 BigKey。

Q8：Redis 和 Memcached 的区别?

对比维度	Redis	Memcached
数据结构	丰富（String/Hash/List/Set/ZSet 等）	仅支持简单的 Key-Value
持久化	支持 RDB 和 AOF	不支持
线程模型	单线程（6.0+ 多线程 I/O）	多线程
集群方案	原生支持 Cluster	需要客户端实现一致性哈希
内存管理	多种淘汰策略	LRU 淘汰
事务支持	支持事务和 Lua 脚本	不支持
发布订阅	支持	不支持
适用场景	复杂数据结构、持久化、分布式锁	纯缓存、简单 KV 存储